

# Attention Mechanisms in NLP: Enhancing Computational Efficiency

|                                     |   |           |
|-------------------------------------|---|-----------|
| Miguel Ara Adánez                   | - | 202102043 |
| Alejandro Martínez de Guinea García | - | 202113492 |
| Lara Ocón Madrid                    | - | 202115710 |
| Javier Prieto Domínguez             | - | 202103783 |
| Javier Rojo Llorens                 | - | 202109117 |

April 2024

## Abstract

This document presents the project proposal for the final project of the courses "Deep Learning" and "Natural Language Processing" by the students of the third year of the Degree in Mathematical Engineering and Artificial Intelligence. The project will span a duration of three weeks during the month of April 2024. Attention mechanisms enable models to focus on specific parts of input data, improving performance on NLP tasks. However, the computational complexity for long sequences remains a challenge. This project aims to address this by exploring various attention mechanisms.

## 1 Introduction

In the realm of deep learning, attention mechanisms [1] have revolutionized the way models process sequences, particularly in natural language processing tasks. These mechanisms have enabled models to focus on specific parts of the input data. However, despite their success, a critical challenge that persists is the computational complexity associated with these mechanisms, especially for long sequences. This project aims to address this challenge by exploring and comparing different techniques designed to make attention mechanisms more computationally efficient.

## 2 Sparse Attention Mechanisms

When looking at reducing the computational cost of performing all the attention operations, a first approach which seems quite logic is to reduce the number of tokens the attention mechanism focuses on. The question at stake is to decide which tokens to select, to maintain the model performance while notoriously reducing the number of comparisons. Although there are several proposed methods to do so which appear to perform at the same level as traditional attention, we will be focusing on understanding and explaining the ones in the following papers:

### 2.1 Big Bird Attention

The combination of three types of sparse attention mechanisms, which seem quite auto explicative: random attention, window attention and global attention. [2]

### 2.2 Sparse Sinkhorn Attention

A proposed method based on dynamic, learnable sorting of internal representations and on differentiable Sinkhorn balancing or normalization. [3]

## 3 Kernelized Attention

Another method that we've found interesting is Kernelized Attention [4]. As we know, the operation to calculate attention value is a dot product followed by a softmax evaluation (non-linear operation). The Kernelized Attention

method approaches this calculation with a different non-linear operation and inverting the order. This is, first it maps queries and keys with a non-linear operation, and then it computes the dot product of these mappings. This way, we can reduce memory-use and computation time, as we can re-use the mappings.

The non-linear operation that we will use is called Random Feature Map [5]. This consists in defining a weight parameter  $w$  (tensor of dimension  $D \times d$ ) that will be trained and define a vector of different dimension using a non-linear function. We've found two derived methods: Trigonometric Random Feature (TRF) and Positive Random Feature (PRF). Both methods will be analysed during the following sessions.

## 4 Linformer: An Efficient Attention Framework

The Linformer, introduced in "*Linformer: Self-Attention with Linear Complexity*" [6], offers a solution to reduce the computational complexity of attention mechanisms from quadratic to linear by approximating the self-attention matrix  $P$  with a low rank alternative. This approach, crucial for handling long sequences efficiently, preserves performance while significantly cutting down on both computational time and memory requirements. In our project, we aim to incorporate the Linformer's technique into our model to explore its efficiency in sentiment analysis tasks. By blending Linformer with our proposed attention strategies, we expect to achieve a sweet spot of efficiency and accuracy, making it a pivotal part of our exploration into creating more scalable and resource-friendly NLP models.

## 5 Locality-sensitivity hashing

A hashing scheme that assigns each vector  $x$  to a hash  $h(x)$  is called locality-sensitive if nearby vectors get the same hash with high probability and distant ones do not. In our case, we actually only require that nearby vectors get the same hash with high probability and that hash-buckets are of similar size with high probability. This process will act as an initial filter to our embeddings and reduce the number of operations needed. [7]

## 6 Proposed Model

Having studied the algorithms proposed, we have decided to implement a mixture of Sparse and Kernelized Attention. With this mechanism we attempt to gain the computation and time efficiency of sparse attention, while reducing the memory usage with kernelized attention, with a probable reduction in the accuracy.

## 7 Data Models and Evaluation

We will use a sentiment analysis dataset that attempts to classify different texts into 6 different categories. The texts have a maximum length of 68 tokens. We picked a dataset that was relatively small (sentence length wise) so we could try more models and compare with more ease. The architecture of the models we will use are encoder-only transformers, with a fully connected layer for classification purposes. Our focus will be on the attention layer of the encoder, where we will substitute the computationally complex attention algorithm for basic transformers with the layers implemented, attempting to increase efficiency while maintaining accuracy.

Since we are tackling a classification model, we will use Cross Entropy Loss for the optimization, and accuracy to compare the different models' performance. We are optimizing efficiency as well, so we will have a look at the time/epoch, trying to reduce it.

## 8 Milestones

The project will be executed following these key milestones:

- **12th of April:**
  - Program some of the algorithms mentioned above.
  - Optimize the algorithms' hyperparameters for maximum accuracy.
  - Compare those implementations with each other and to the one proposed in class.

- **19th of April:**

- Program the improvement on the methods proposed in the literature.
- Compare it with the mechanisms already programmed.
- Have individual mathematical demonstrations of the improvement of the mechanisms.

## References

- [1] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., & Polosukhin, I. (2017). *Attention is all you need*. <https://arxiv.org/abs/1706.03762>
- [2] Zaheer, M., Guruganesh, G., Dubey, A., Ainslie, J., Alberti, C., Ontanon, S., Pham, P., Ravula, A., Wang, Q., Yang, L., & Ahmed, A. (2020). *Big Bird: Transformers for Longer Sequences*. <https://arxiv.org/abs/2007.14062>
- [3] Tay, Y., Bahri, D., Yang, L., Metzler, D., & Juan, D. (2020). *Sparse sinkhorn attention*. <https://arxiv.org/abs/2002.11296>
- [4] Luo, S., Li, S., Cai, T., He, D., Peng, D., Zheng, S., Ke, G., Wang, L., & Liu, T. (2021). *Stable Fast and Accurate: Kernelized Attention with Relative Positional Encoding*. <https://arxiv.org/abs/2106.12566>
- [5] Peng, H., Pappas, N., Yogatama, D., Schwartz, R., Smith, N. A., & Kong, L. (2021). *Random feature attention*. <https://arxiv.org/abs/2103.02143>
- [6] Wang, S., Li, B. Z., Khabza, M., Fang, H., & Ma, H. (2020). *Linformer: Self-Attention with Linear Complexity*. <https://arxiv.org/abs/2006.04768>
- [7] Kitaev, N., Kaiser, L., & Levskaya, A. (n.d.). *Reformer: the efficient transformer*. <https://openreview.net/forum?id=rkgNKkHtvB>